

suggu

CLI Project Plan & Architecture — the master document for building v1.0.0

Owner: Shubham (.NET developer) · Working directory: D:\suggu · Status: design locked, implementation not started · Date: 2026-07-09

This document is self-contained. A human reading it end-to-end understands what suggu is, how it is structured, and in what order it gets built. An AI assistant given this document in a fresh session has everything needed to continue the work without further explanation — see Section 11 for AI-specific instructions. The companion file SUGGU-PROJECT.md in the repository root holds the long-term vision (MCP server, IDE integration); this document deliberately covers only the CLI.

1. What suggu is and why it exists

suggu is a personal, deterministic scaffolding and verification CLI for .NET clean-architecture projects. When an AI coding agent (Claude Code) works on a .NET solution, a large share of its effort is burned on repetitive, rule-based work: creating the same folder trees in every layer, writing near-identical repository interfaces for every entity, installing and wiring the same NuGet packages per layer, and generating the same CQRS command/handler/validator shapes for every feature. That work is deterministic (fully derivable from rules), repetitive (same shapes in every project), and error-prone when hand-written (namespace drift, missed DI registrations, inconsistent naming).

suggu owns the deterministic 90%; the AI fills the semantic 10%.

The durable value is not just token savings: it is consistency (identical output every time, forever), enforcement (the tool can verify a solution still conforms), and context compression (one command tells a human or an AI what the whole solution looks like). suggu is built first as a daily personal tool; every design decision favors being genuinely useful this week over being impressive later.

2. Scope of v1.0.0 — what the CLI must do

Six capabilities define v1. Everything else (MCP server, Visual Studio integration, AI features, other ecosystems) is explicitly out of scope for v1.

#	Capability	Example
1	Scaffold a complete project skeleton: clean architecture, api or mvc, target framework 8/9/10, all layers, folders, packages, seed files. More architectures (onion, vertical slice) come later as data, not code.	suggu new Shop --arch clean --type api --framework 9
2	Set up and verify each layer against a defined structure: which folders exist, which packages are installed in which layer, per framework version. The rules are provided by the owner and stored as data.	suggu setup application suggu check layers
3	Inspect the solution: folder tree per layer, installed packages, project references.	suggu info application suggu info packages application
4	Create folders (single or multiple) in a chosen layer.	suggu add folder Specifications Aggregates --layer domain

#	Capability	Example
5	Generate repository interfaces by scanning Domain/Entities/** recursively and mirroring the folder structure under Domain/Interfaces (Product.cs → IProductRepository.cs).	suggu add repositories
6	Generate per-feature CQRS scaffolds — command, handler, validator, DTO — in the right Features subfolder, from owner-defined templates that match how the owner codes. The same mechanism powers future generators for all four layers.	suggu add command GetPublicInfo --parent User

2.1 Command grammar (locked)

Consistent verb–noun shape. Flags accept both --flag value and --flag=value. Every command supports --json, --dry-run, and --force where applicable.

```
suggu new <Name> --arch clean --type api|mvc --framework 8|9|10 [--db
pgsql|sqlserver]
suggu setup <layer> # folders + packages for one layer, from the pack
suggu add folder <Name...> --layer <layer>
suggu add repositories # entity scan → mirrored interfaces
suggu add command <Name> --parent <Module>
suggu add query <Name> --parent <Module>
suggu info solution | info <layer> | info packages <layer>
suggu check layers | check packages
suggu doctor # aggregate of all checks (v1.0.0)
```

3. Solution structure — three projects, no more

Resist premature layering. Suggu.Core is organized as folders that may become assemblies later if and only if two consumers need them. The default knowledge pack (data + templates) ships as embedded resources inside Suggu.Core.

Project	Responsibility	Rules
Suggu.Cli	Command definitions, argument parsing, console rendering (trees, tables, colors).	Thin. No business logic. Each command parses input, calls Core, renders the result.
Suggu.Core	Everything else, as folders: Workspace (discovery, suggu.json) · Packs (schema, loader, embedded default) · Planning (plan model + executor) · Generation (templates, entity scanning) · Inspection (csproj/package/reference reading).	No console I/O. Returns structured results. This is the assembly a future MCP server wraps unchanged.
Suggu.Tests	Plan assertions (no disk), integration tests against temp-dir fixture solutions, snapshot tests of generated file trees.	Integration-heavy by design; a scaffolding tool is judged by the files it writes.

Dependency direction: Suggu.Cli → Suggu.Core. Never the reverse. Any future frontend (MCP, editor extension) also points only at Core. Business logic is never duplicated across frontends.

4. Pillar 1 — Knowledge Packs (architecture as data)

Nothing about "clean architecture" is hardcoded in the engine. A pack is a versioned data bundle (JSON/YAML plus template files) that fully describes one architecture profile. The engine is generic; the pack is law. Adding onion or vertical-slice support later means writing a new pack — not refactoring the tool. Adding .NET 11 support means editing a data row.

4.1 What a pack contains

Section	Contents	Consumed by
meta	Pack name (clean-webapi), schema version, description, supported frameworks (8/9/10), supported project types (api/mvc), supported databases (pgsql/sqlserver).	loader, new, --help
layers	For each layer (Domain, Application, Infrastructure, Api): detection patterns (*.Domain; *.Infra or *.Infrastructure), allowed project references between layers (Domain references nothing; Application → Domain; Infrastructure → Application + Domain; Api → Application + Infrastructure).	discovery, check, doctor
folders	The canonical folder tree per layer — the owner-defined structure (e.g. Domain: Common, Entities, Interfaces, ValueObjects, Enums, DomainEvents, Exceptions, Aggregates, Specifications; Application: Features, DTOs, Contracts, Behavior, DependencyInjection; etc.). Empty folders are persisted via <Folder Include> entries in the .csproj, not .gitkeep.	setup, new, check layers
packages	Per layer × per framework × per database: exact package lists with version ranges (e.g. Infrastructure + net9 + pgsql → Npgsql.EntityFrameworkCore.PostgreSQL 9.x, Dapper 2.x). Domain has no packages by design.	setup, new, check packages
seeds	Template files dropped by new/setup: BaseEntity, AuditableEntity, IAggregateRoot, Result, ValidationBehavior, ExceptionHandlingMiddleware, ApiResponse, DependencyRegistration per layer, and so on. A pack is folders + packages + seed files, never folders alone.	new, setup
generators	Named scaffold recipes — the mechanism behind add command / add query / add repositories and every future add-style command. See 4.2.	add *
rules	Validation rules for check/doctor: required folders present, forbidden references (Domain must not reference EF Core), naming conventions, entity→repository coverage.	check, doctor

4.2 Generators — the extensibility mechanism

A generator is a pack entry, not a C# command. Each generator declares: a name (command, query, repository, endpoint ...), its parameters (Name, Parent, ...), the target layer, and a list of outputs — each output being a path pattern plus a template reference. Example, generator "command":

outputs:

Features/{Parent}/Commands/{Name}/{Name}Command.cs	→ command.sbn
Features/{Parent}/Commands/{Name}/{Name}Handler.cs	→ handler.sbn
Features/{Parent}/Commands/{Name}/{Name}Validator.cs	→ validator.sbn
Features/{Parent}/Commands/{Name}/{Name}Dto.cs	→ dto.sbn

One generic engine reads any generator entry, renders templates, and emits a plan (Pillar 2). "suggu add command GetPublicInfo --parent User" navigates to the Application layer's Features folder, creates the

User module folder if missing, and generates all four files with the owner's own coding style baked into the templates. When a new repetitive shape appears next month, the owner adds a generator entry plus templates to the pack — zero new tool code. Special case: "add repositories" is a generator whose input is not CLI parameters but a scan (every entity found under Domain/Entities/**), producing one output per entity with the subfolder structure mirrored.

4.3 How to set Pillar 1 up (implementation approach)

- Define the pack schema first as plain C# record types in Suggu.Core/Packs — the schema IS the contract. Stamp a schemaVersion field from day one.
- Write the default clean pack as JSON + template files under Suggu.Core/Packs/Default/, marked as embedded resources. The loader reads embedded-by-name or from-disk-by-path through one code path, so external user packs are free later.
- Templates use plain placeholder substitution ({Name}, {Parent}, {Namespace}, {RootNamespace}) until a template needs loops or conditionals; then switch to Scriban (expected around the generator work, v0.4). Generated code style: file-scoped namespaces, nullable enabled.
- The reference for the default pack's layer shape is the TaskFlow-CleanArchitecture-API repo analyzed in SUGGU-PROJECT.md §2 — but the pack is law, the repo is only inspiration: enforce exact Entities→Interfaces mirroring even where the reference repo is inconsistent.
- Repository interfaces extend a generic IRepository<T> generated once — per-entity files stay nearly empty, which makes regeneration safe.

5. Pillar 2 — Plan → Apply (one execution pipeline)

Every command, no exceptions, works in two stages. Stage one (Plan): inspect the workspace and produce a list of intended operations — no disk writes. Stage two (Apply): one shared executor applies any plan with uniform behavior. This is what prevents logic duplication structurally rather than by discipline.

5.1 The operation vocabulary

Operation	Meaning
CreateFolder	Create a directory (and its csproj entry when it will stay empty).
WriteFile	Render a template with a model and write the result. Carries source template id + target path.
AddPackage	Install a NuGet package (name + version range) into a specific project.
AddReference	Add a project-to-project reference.
EditCsproj	A structured csproj change (property, item) — never regex over XML text.
CreateProject / CreateSolution	Wrap dotnet new / dotnet sln; used by suggu new.

The vocabulary is closed and small on purpose. If a future feature seems to need a new operation type, first check whether it composes from existing ones.

5.2 The executor — the only code that touches the disk

- Idempotency: existing files are skipped by default; `--force` overwrites. Running any generator twice must never clobber customized files.
- `--dry-run` for free on every command: print the plan, apply nothing.
- Uniform reporting: every command returns the same report shape — created / skipped / overwritten / failed lists plus diagnostics — rendered as human output by default and as stable JSON with `--json`.
- Ordering and safety: folders before files, projects before packages; a failed operation stops the run and reports exactly what was and wasn't done.

Composition is the payoff: "suggu setup application" builds one layer's plan; "suggu new" concatenates the solution plan plus all four layer plans and applies once; "suggu add repositories" is a scan that emits WriteFile operations. Skip/overwrite/reporting/dry-run logic exists in exactly one place forever. Testing gets trivial: most tests assert on the plan object and never touch the disk.

5.3 How to set Pillar 2 up (implementation approach)

- Model the plan as an ordered list of operation records in Suggu.Core/Planning. Operations are data (target path, template id, package id + version), not delegates — a plan must be printable, serializable, and diffable.
- Write the executor with a dry-run flag from day one, and give it a single ExecutionReport return type. The CLI renders that type; the JSON output serializes it. One report shape, decided once.
- Commands never call File.WriteAllText or Process.Start themselves — grep for such calls in code review; outside the executor they are a bug.
- Wrap dotnet CLI invocations (dotnet new, dotnet add package, dotnet sln add) behind a small process runner inside the executor so output capture, error handling, and logging are uniform.

6. Workspace discovery and suggu.json

Before any command runs, suggu answers "where am I and what does this solution look like": walk up from the current directory to find the .sln or .slnx (never assume the tool runs at solution root); detect layers by the pack's naming patterns; parse each .csproj for target framework, packages, and references. The result is persisted to suggu.json at the repository root, which then becomes the source of truth — and the escape hatch for non-standard layouts (src/ folders, a layer named Core instead of Domain): edit the file, and suggu believes it.

suggu.json holds at minimum: schema version, pack/profile name, the layer map (layer name → project path), and style options (interfaces folder name, generic repository yes/no). Keep it small. The info commands are discovery printed: "info application" renders that layer's folder tree, packages, and references; "info packages application" is a slice of the same data. Discovery feeds everything — generators need to know where the Application layer is; new needs to know the directory is empty; check compares discovery against the pack.

7. Build order — v0.1 to v1.0.0

Each step ships something usable the same week and is the foundation of the next. Note the deliberate inversion: suggu new is built last, not first — it demos best but runs least often, and by v0.5 it is ~90% composition of parts that already exist and are tested.

Version	Deliverable	Done when
v0.1 skeleton + discovery	CLI shell (Spectre.Console.Cli), workspace discovery, suggu.json read/write, info solution / info <layer> / info packages, add folder (single + multiple).	Run suggu info on an existing clean-architecture solution and get a correct layer map, tree, and package list. Discovery survives .sln and .slnx, and non-root working directories.
v0.2 plan/apply + the star	Plan model + executor (idempotent, --dry-run, --force, ExecutionReport, --json). add repositories: recursive entity scan, mirrored interface generation, IRepository<T> base. Package as global dotnet tool now (dotnet tool install -g suggu).	Running add repositories twice reports "created 0, skipped N". Generated files compile inside the target solution. The tool is installed globally and used on a real project.
v0.3 the pack	Pack schema (versioned), embedded default clean pack: full folder trees, package tables for net8/9/10 × pgsql/sqlserver, seed templates. Loader (embedded or disk). check layers / check packages.	check commands report real drift on a deliberately broken fixture solution. Adding a framework version is demonstrably a data-only change.
v0.4 setup + generators	setup <layer> (folders + packages + seeds from pack, through the executor). Generator engine reading pack generator entries; add command / add query producing command, handler, validator, DTO under Features/{Parent}/...; Scriban adopted for templates.	add command GetPublicInfo --parent User produces four correctly-namespaced files matching the owner's coding style, idempotently. A new generator can be added without touching C#.
v0.5 suggu new	Full project scaffolding: solution, four projects, references, folders, packages, seeds — composed from setup plans + CreateProject/CreateSolution operations, honoring --arch/--type/--framework/--db.	suggu new Shop --arch clean --type api --framework 9 --db pgsql produces a solution that builds (dotnet build) with zero manual fixes.
v1.0.0 hardening	doctor (aggregate of all checks + entity→repository coverage), polish on reports and dry-run, pack schema documentation, README with the full command reference.	doctor on a healthy solution is green; on each seeded defect it reports the specific rule violated. Documentation is complete enough to onboard from scratch.

8. Locked decisions

Settled deliberately; do not relitigate without a strong reason. Changing any of these later is expensive.

Decision	Choice	Rationale
Command parsing	Spectre.Console.Cli	Stable API; Spectre.Console provides tree/table rendering for info commands nearly for free.
Template engine	Placeholders → Scriban at v0.4	Plain substitution is enough for seeds and repositories; CQRS generator templates need conditionals.
Output contract	Human default, --json everywhere	Daily interactive tool first; agents pass --json. One report type keeps the JSON shape stable.
Idempotency	Skip existing; --force to overwrite	Regeneration must never destroy customized files. Executor-enforced, not per-command.
Code style	File-scoped namespaces, nullable on, IRepository<T> base	Matches the owner's style; near-empty generated files make regeneration safe.
Empty folders	<Folder Include> in csproj	The reference template's own convention; no .gitkeep litter.
Solution formats	.sln and .slnx	Both exist in the wild from .NET 9 onward.
Distribution	Global dotnet tool from v0.2	Distribution friction kills feedback loops; CI usage becomes trivial.
Layer naming	Detect *.Api/*.Domain/*.Application/*.Infra(structure); suggu.json overrides	Convention covers ~90% of repos; the file is the escape hatch for the rest.

9. Non-goals for v1 (postponed on purpose)

- No MCP server until the CLI has real daily usage — usage reveals which tools/resources actually matter. The Core/CLI split already guarantees the MCP server will be a thin wrapper when its time comes.
- No Roslyn dependency in v1. Entity scanning works on syntax/convention (public classes under Domain/Entities/**). Roslyn semantic analysis (find-by-base-class, safe DI-registration insertion, reference graphs) is the headline upgrade of the next major phase.
- No AI features. Where AI enters later, it produces structured plans that the deterministic engine validates and executes — AI never writes files directly.
- No Visual Studio tool window, no TUI, no other ecosystems (Node, Java, ...). suggu is .NET-only by identity.
- No general package-compatibility oracle. Compatibility knowledge covers exactly the packages the packs bless, per framework — plus live NuGet queries for versions. Nothing more.

10. Known risks and their mitigations

Risk	Mitigation
Pack maintenance treadmill: every .NET release and package major bump requires pack updates; stale packs are worse than no tool.	Keep the blessed surface small — one excellent default pack beats five decaying ones. Pack updates are data-only by design. Schema is versioned so old packs fail loudly, not silently.

Risk	Mitigation
Over-abstraction: building six "engines" and a plugin system before there are features to fill them.	Three projects only. Folders inside Core. Extract an assembly only when two consumers need it. Packs stay data; rules that need logic become named C# rules referenced by id — no YAML DSL for code.
Idempotency bugs silently destroying customized files.	Skip-by-default enforced in the executor, not per command; snapshot tests re-run every generator twice and assert zero changes on the second run.
Grammar drift: flags and verbs accreting inconsistently as commands are added.	Section 2.1 is the contract. New commands must fit verb–noun + standard flags or the grammar is revised deliberately, in this document first.

11. Instructions for an AI assistant reading this document

If you are Claude (or another AI assistant) and were given this PDF in a fresh session: this is the master plan for suggu, a .NET CLI being built incrementally in D:\suggu by Shubham. Do the following, in order. **(1)** Read SUGGU-PROJECT.md in the repository root for the long-term vision if present; this PDF overrides it wherever they disagree about the CLI. **(2)** Inspect the current state of the code in D:\suggu — check which projects exist (Suggu.Cli / Suggu.Core / Suggu.Tests) and which commands already work — to determine which roadmap version (Section 7) is in progress. Do not re-scaffold or redesign what already exists. **(3)** Continue from the first unmet "Done when" criterion in the Section 7 table, respecting the two pillars: all architecture knowledge lives in the pack (Section 4), and all disk mutations flow through the plan/apply executor (Section 5) — code that writes files outside the executor is a defect. **(4)** Honor every locked decision in Section 8 and every non-goal in Section 9 without re-opening them; if a change seems necessary, say so explicitly and get agreement before deviating. **(5)** Keep Suggu.Cli thin (parse → call Core → render) and keep Suggu.Core free of console I/O. **(6)** When adding a scaffold shape, prefer adding a pack generator entry over adding C# code. Work incrementally: small steps, each leaving the tool usable.

Compass heading: v0.1 makes suggu see the solution · v0.2 makes it generate · v0.3 makes it know · v0.4 makes it scaffold features · v0.5 makes it create projects · v1.0.0 makes it enforce. Each version is useful on its own even if work stops there.